

Internet of Things (IoT) and Embedded

Unit 7

1 Introduction

The Internet of Things (IoT) refers to a network of interconnected physical objects that can communicate and exchange data with each other over the internet. These objects, embedded with sensors, software, and other technologies, collect and transmit data, enabling intelligent systems to interact with the physical world.

As of recent estimates, the number of IoT devices has far surpassed the global human population. For example, in 2021, there were more than 40 billion IoT devices compared to the world population of around 7.8 billion. This trend is expected to accelerate, with projections suggesting that by 2025, there could be over 80 billion connected devices globally.

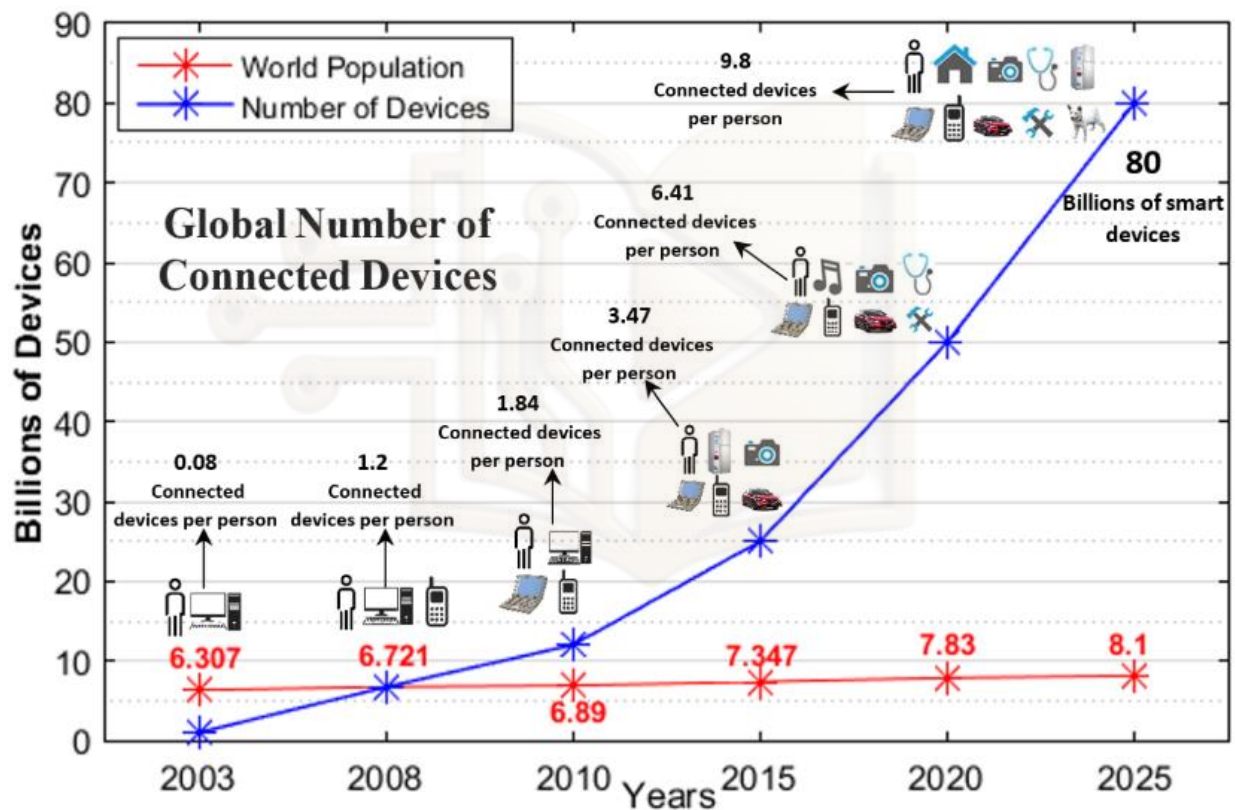


Figure 1-1: IoT devices projection vs Population

The proliferation of IoT devices is driven by advancements in technology and increasing demand for smart solutions in various sectors, including healthcare, manufacturing, transportation, and home automation. As IoT continues to evolve, it is anticipated to bring even more innovative applications and significant improvements in quality of life, efficiency, and productivity.

Internet of Things (IoT) and Embedded

Unit 7

1.1 History of IoT

The era of the Internet of Things (IoT) is widely recognized to have begun between 2008 and 2009 when the number of devices connected to the Internet surpassed the global human population. This milestone marked the onset of a new age where "things" were more connected than people, heralding a significant technological shift. The term "Internet of Things" was coined in 1999 by Kevin Ashton, who was working at Procter & Gamble at the time. Ashton introduced this phrase to describe the concept of linking the company's supply chain to the Internet, envisioning a system where objects could communicate and share data autonomously.



Figure 1-2: Kevin Ashton

Ashton further elaborated that IoT involves adding senses to computers. In the twentieth century, computers were akin to "brains without senses," processing only the information that humans input through methods like typing and barcodes. However, in the twenty-first century, IoT has revolutionized this paradigm by enabling computers to sense their environment independently. This advancement allows devices to collect, interpret, and act on data without human intervention, fundamentally transforming our interaction with technology. In simple word ***the term IoT, or Internet of Things, refers to the collective***

Internet of Things (IoT) and Embedded

Unit 7

network of connected devices and the technology that facilitates communication between devices and the cloud, as well as between the devices themselves.

The transition to IoT represents a major technological shift, creating a tighter integration between the physical world and digital systems. This integration has led to significant improvements in efficiency, accuracy, and automation across various applications, from smart homes to industrial automation and healthcare, highlighting the profound impact of IoT on modern life.

1.2 General Architecture of IoT system

The Internet of Things (IoT) architecture typically consists of four fundamental layers: **Sensing Layer**, **Network Layer**, **Data Processing Layer**, and **Application Layer**. Each layer has distinct functions and roles, forming a cohesive framework for IoT systems.

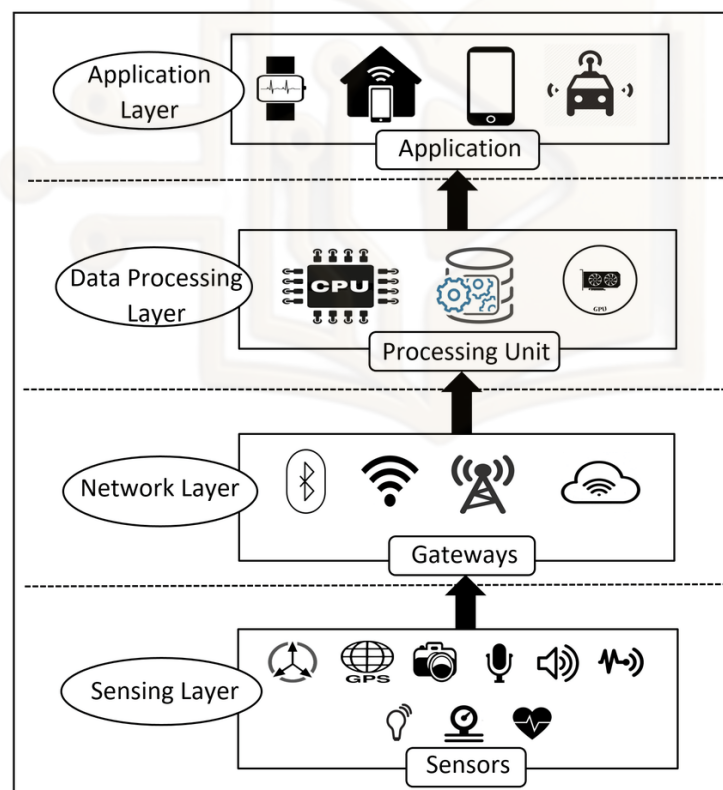


Figure 1-3: Architecture of IoT

1. Sensing Layer

The sensing layer forms the foundation of the IoT system, responsible for collecting data from the physical world and initiating actions based on commands from higher layers. Sensors capture environmental data, while actuators perform tasks such as turning on lights or adjusting machinery based on instructions received from controllers.

This layer encompasses physical devices equipped with sensors and actuators, along with controllers that manage these devices.

Key Functions:

- **Data Acquisition:** Sensors detect and measure parameters such as temperature, humidity, motion, or light.
- **Device Interaction:** Acts as the interface between the physical environment and the IoT ecosystem.
- **Device Control:** Devices like actuators perform tasks based on received instructions (e.g., turning on lights, adjusting a thermostat).

Components:

- Sensors (e.g., temperature, proximity, pressure sensors)
- Actuators (e.g., motors, switches)
- IoT-enabled devices (e.g., smart appliances)

2. Network Layer

The **Network Layer** handles the seamless transfer of data between the sensing layer and other components of the IoT ecosystem, often using a variety of communication protocols.

Key Functions:

- **Data Transmission:** Transfers collected data from sensors to data processing units or cloud servers.
- **Protocol Management:** Ensures data is transmitted securely and efficiently using protocols like MQTT, HTTP, and CoAP.

Internet of Things (IoT) and Embedded

Unit 7

- **Connectivity:** Provides both local and wide-area network connectivity through wired or wireless technologies.

Components:

- **Communication Technologies:** Wi-Fi, ZigBee, LoRa, GSM/GPRS, and Ethernet.
- **IoT Gateways:** Bridge devices that connect local networks to the internet or cloud.
- **Network Protocols:** Protocols like MQTT, CoAP, and HTTP ensure reliable data exchange.

3. Data Processing Layer

The **Data Processing Layer** is responsible for aggregating, filtering, and analyzing raw data collected from the sensing layer to extract meaningful insights.

Key Functions:

- **Data Aggregation:** Combines data from multiple sources for unified analysis.
- **Data Filtering:** Removes noise or irrelevant data to improve efficiency.
- **Data Analytics:** Performs real-time or batch processing to extract actionable insights.

Components:

- **Edge Devices:** Perform localized data processing to reduce latency.
- **Cloud Servers:** Store and analyze large-scale IoT data.
- **Machine Learning Models:** Provide predictive analytics and intelligent decision-making.

4. Application Layer

The Application Layer represents the user-facing interface, enabling interaction with the IoT system for monitoring and control.

Key Functions:

- **Visualization:** Presents processed data in user-friendly dashboards.

- **User Interaction:** Allows users to monitor and control devices via web or mobile applications.
- **Service Delivery:** Provides IoT-enabled services like home automation, smart energy management, or predictive maintenance.

Components:

- **User Interfaces:** Mobile apps, web dashboards, and APIs.
- **Application Protocols:** RESTful APIs and JSON for communication.
- **Service Platforms:** Smart home systems, healthcare monitors, and industrial automation tools.

1.3 Key Roles of Embedded Systems in the IoT Domain

Embedded systems play a pivotal role in enabling the seamless operation of IoT devices and networks. These systems act as the backbone of IoT infrastructure by integrating hardware and software to perform specific tasks efficiently and reliably. The key roles of embedded systems in the IoT domain include:

1. Data Collection and Processing

Embedded systems manage sensors that collect data from the physical environment, such as temperature, humidity, pressure, and motion. They preprocess and filter the data to ensure only relevant information is transmitted, optimizing bandwidth and reducing power consumption.

2. Real-Time Operation

IoT applications often require real-time responses. Embedded systems ensure low-latency performance for time-sensitive tasks, such as controlling industrial machinery, automating home devices, or triggering emergency alerts.

3. Device Control

Internet of Things (IoT) and Embedded

Unit 7

Embedded systems control actuators that perform specific actions based on data received or predefined instructions. For example, in smart homes, embedded systems manage lighting, heating, and security systems.

4. Communication Interface

Embedded systems enable IoT devices to communicate with each other and with central networks using wireless protocols like Bluetooth, ZigBee, Wi-Fi, LoRa, or GSM. They ensure reliable and secure data transmission across the IoT ecosystem.

5. Energy Efficiency

Power consumption is critical for IoT devices, especially those running on batteries. Embedded systems are designed to optimize energy usage, ensuring extended operation in resource-constrained environments.

6. Security

IoT devices are vulnerable to cyber threats. Embedded systems implement encryption, authentication, and secure boot mechanisms to safeguard data and prevent unauthorized access.

7. Scalability and Interoperability

Embedded systems facilitate seamless integration and scalability of IoT networks by supporting a wide range of devices and protocols. This ensures interoperability among diverse IoT ecosystems.

8. Edge Computing

By processing data locally, embedded systems reduce dependency on cloud infrastructure. This minimizes latency, enhances security, and optimizes network resources for IoT applications requiring immediate decisions.

9. Customizability

Embedded systems are highly customizable to suit the specific needs of IoT applications. They can be tailored to support diverse use cases, from smart agriculture to industrial automation.

10. Cost Efficiency

Embedded systems are designed to be cost-effective, making IoT solutions affordable and scalable for both consumer and industrial markets.

2 Common IoT communication Protocols

In the realm of Internet of Things (IoT), protocols serve as essential frameworks that facilitate seamless communication and interoperability among interconnected devices and systems. These protocols are specifically tailored to address the unique challenges posed by IoT environments, such as constrained resources, varying network conditions, and diverse device capabilities. Here's a detailed overview of three common IoT protocols: MQTT, CoAP, and HTTP.

1. MQTT (Message Queuing Telemetry Transport) Protocol

MQTT is a lightweight publish-subscribe protocol built on top of TCP/IP, designed for efficient communication in IoT environments. MQTT employs a message broker architecture where senders (publishers) publish messages and receivers (subscribers) interested in specific topics receive these messages. This decouples data producers and consumers, allowing them to communicate through shared topics without needing to know each other directly. This architecture is particularly suited for IoT applications where devices need to exchange data reliably and efficiently.

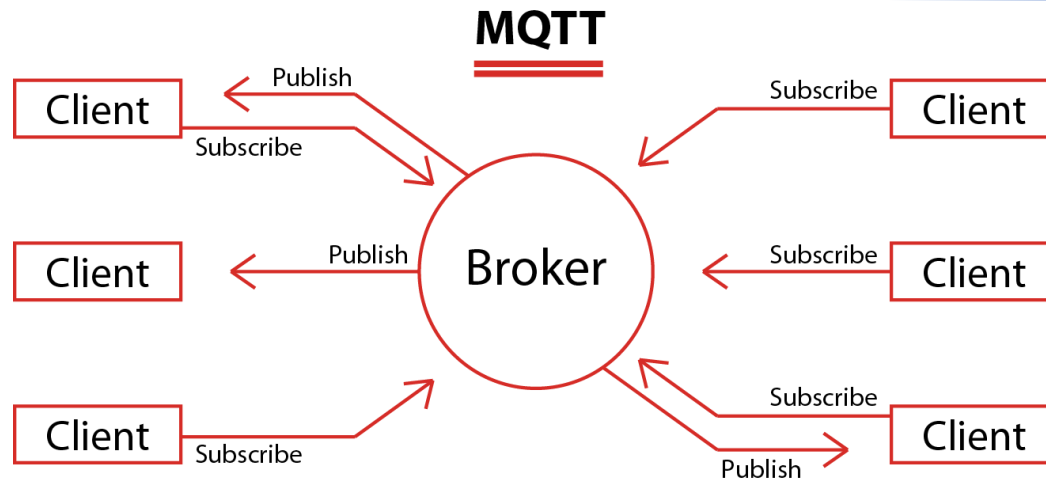


Figure 2-1: MQTT Protocol: Two or unique devices subscribed or publish to same topic

- **Purpose:** MQTT is a lightweight messaging protocol designed for efficient communication in IoT applications with low bandwidth and high latency or unreliable networks.
- **Usage Scenario:** It is widely used in scenarios where devices need to publish and subscribe to data streams, such as sensor data and control messages.
- **Key Features:**
 - **Lightweight:** MQTT is designed to be lightweight, both in terms of network bandwidth and computational overhead. This makes it ideal for resource-constrained devices often used in IoT, where limited data transmission capabilities and processing power are available.
 - **Publish/Subscribe Model:** MQTT follows a publish/subscribe model, allowing clients to publish messages to a topic and other clients to subscribe to these topics. This model enables efficient communication between devices and reduces the need for direct connections between devices, making it suitable for large-scale IoT networks.
 - **Reliability and Fault Tolerance:** MQTT uses a quality of service (QoS) mechanism to ensure message delivery, even in cases where the network connection may be unreliable. The QoS levels include:
 - **QoS 0:** At most once delivery – messages are delivered at most once.
 - **QoS 1:** At least once delivery – messages are delivered at least once, with retransmission in case of network failure.
 - **QoS 2:** Exactly once delivery – messages are delivered exactly once, using a two-step handshake process.

- **Efficient Bandwidth Usage:** Due to its lightweight protocol design, MQTT is efficient in terms of bandwidth usage, which is crucial for IoT applications where devices may be operating over low-bandwidth, high-latency networks.
 - **TCP/IP-Based Communication:** MQTT operates over the TCP/IP protocol, which ensures reliable communication and easy integration with existing network infrastructures. This makes it compatible with various network environments, including Wi-Fi, cellular, and low-power wide-area networks (LPWAN).
 - **Support for Disconnected Operation:** MQTT supports the ability to store messages when a client is temporarily disconnected and then deliver them once the client reconnects. This feature is particularly useful in IoT environments where devices may frequently lose network connectivity.
 - **Flexibility and Scalability:** The MQTT protocol is scalable, making it suitable for applications ranging from a few devices to millions of devices. It allows for easy expansion and adaptation to different IoT use cases without major changes to the protocol or infrastructure.
 - **Security:** MQTT provides security features such as message encryption (using SSL/TLS), authentication, and authorization mechanisms, ensuring that data is transmitted securely across the network. This is crucial for IoT applications where sensitive data is exchanged between devices.
- **Applications:** MQTT is suitable for remote monitoring, telemetry, and real-time control applications in industries like manufacturing, agriculture, and smart cities.

3 Common IoT Hardware Platform

The success of an IoT project depends heavily on selecting the appropriate hardware platform that aligns with the project's goals, complexity, and resource requirements. There are three most popular IoT platforms—**Arduino**, **ESP32**, and **Raspberry Pi**—detailing their features, applications, and strengths while providing guidance on their optimal use cases. By understanding the unique capabilities of each platform, developers can make informed decisions to meet the functional and operational needs of IoT systems, whether for simple prototypes or advanced edge-computing solutions.

3.1 Arduino

Arduino is an open-source hardware and software platform that simplifies the process of building electronics projects. Known for its ease of use and versatility, it empowers hobbyists, students, and professionals alike to prototype and develop embedded systems quickly. The platform includes a variety of microcontroller boards and a straightforward Integrated Development Environment (IDE), making it a go-to choice for beginners and experts.

Arduino was first developed in 2005 at the Interaction Design Institute Ivrea in Italy by Massimo Banzi, David Cuartielles, Tom Igoe, Gianluca Martino, and others. It was initially created to provide a low-cost and simple way for students and designers to work on interactive projects. One of Arduino's greatest strengths is its vast online community. From open-source libraries to forums and project repositories, users can find extensive resources to learn and build projects. Its ecosystem supports both hobbyists and professionals in scaling their ideas.

3.1.1 Types of Arduino Boards

Arduino offers a wide range of boards catering to various requirements, from basic prototyping to advanced applications in IoT, robotics, and AI. Some popular Arduino boards include:

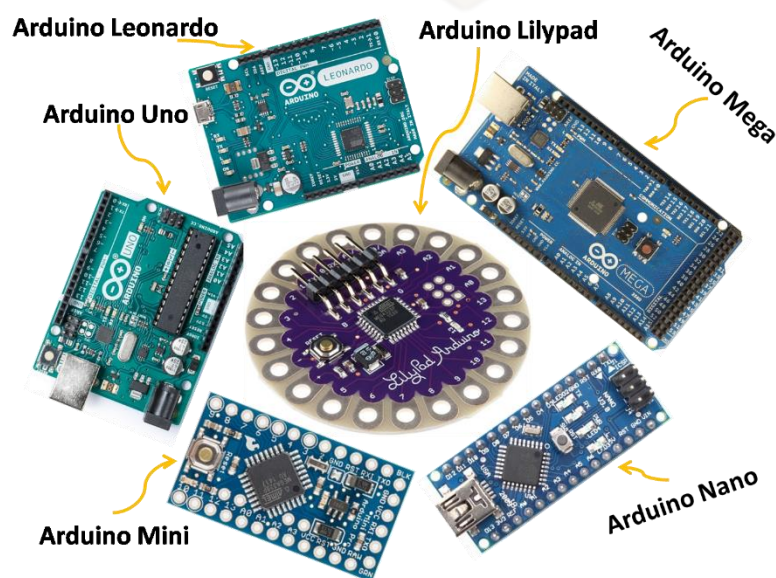


Figure 3-1: Types of Arduino

Internet of Things (IoT) and Embedded

Unit 7

- **Arduino Uno:** Ideal for beginners and most general-purpose projects.
- **Arduino Mega:** Offers more I/O pins and memory, perfect for complex projects.
- **Arduino Nano:** A compact board suitable for space-constrained designs.
- **Arduino Pro Mini:** A minimalist version for permanent installations.
- **Arduino Leonardo:** Supports USB communication directly with the microcontroller.
- **Arduino MKR Series:** Tailored for IoT projects, including WiFi and cellular connectivity.
- **Arduino Nano 33 BLE Sense:** Equipped with sensors and Bluetooth for AI/ML applications.

3.1.2 Detailed Features of Arduino Uno

The **Arduino Uno**, based on the ATmega328P microcontroller, is one of the most popular boards in the Arduino family. Its simplicity and flexibility make it a favorite for beginners and experts.

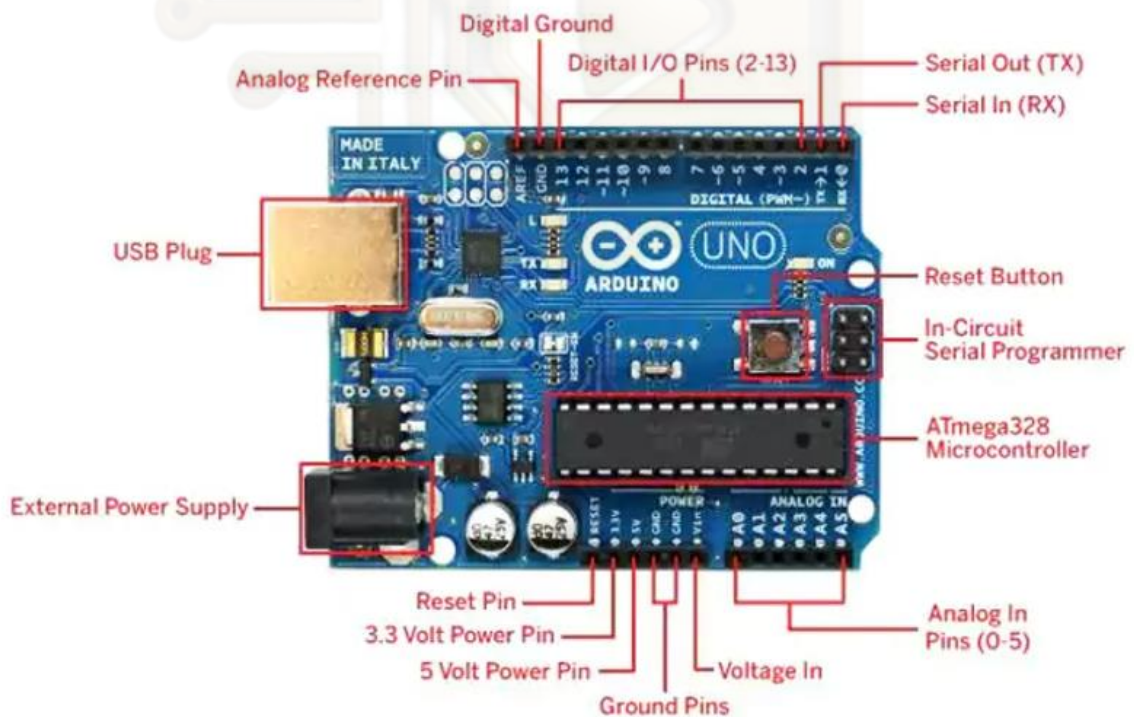


Figure 3-2: Arduino Uno Pin diagram

Key Features:

- **Microcontroller:** ATmega328P.
- **Operating Voltage:** 5V.
- **Input Voltage:** 7–12V (via barrel jack).
- **Digital I/O Pins:** 14 (6 PWM capable).
- **Analog Input Pins:** 6.
- **Clock Speed:** 16 MHz.
- **Flash Memory:** 32 KB (with 0.5 KB reserved for the bootloader).
- **SRAM:** 2 KB.
- **EEPROM:** 1 KB.
- **Connectivity:** USB-B port for programming and power supply.
- **Power Options:** USB, external adapter, or battery.
- **Programming:** Compatible with the Arduino IDE, which supports C/C++ programming languages.

Capabilities:

- **Easy Integration with Sensors and Actuators:** Simplifies connecting components like temperature sensors, LEDs, and motors.
- **Prototyping:** Ideal for rapid prototyping and testing embedded systems.
- **Compatibility with Shields:** Expansion boards (shields) like motor drivers, WiFi, and GSM make it versatile.
- **Wide Community Support:** Numerous libraries, forums, and tutorials available for easy learning.

3.2 ESP32

The ESP32 is a powerful, low-cost, and highly versatile microcontroller platform designed for IoT applications. Manufactured by Espressif Systems, it is known for its integrated WiFi and Bluetooth capabilities, making it ideal for wireless communication projects. The ESP32 is widely appreciated for its robust performance, low power consumption, and rich feature set, including dual-core processing, built-in sensors, and extensive GPIO pins.

Internet of Things (IoT) and Embedded

Unit 7

First released in 2016, the ESP32 has rapidly become a preferred choice for IoT and edge-computing projects. Its development ecosystem includes the Arduino IDE, Espressif's own ESP-IDF framework, and MicroPython, providing flexibility for developers with different preferences and levels of expertise.

Like Arduino, the ESP32 benefits from a large online community, offering support through forums, libraries, and tutorials. It is particularly favored for projects that require wireless connectivity and edge processing capabilities.

3.2.1 Types of ESP32 Boards

The ESP32 family includes various boards, each designed to address specific needs in IoT and embedded applications. Some of the most commonly used boards are:

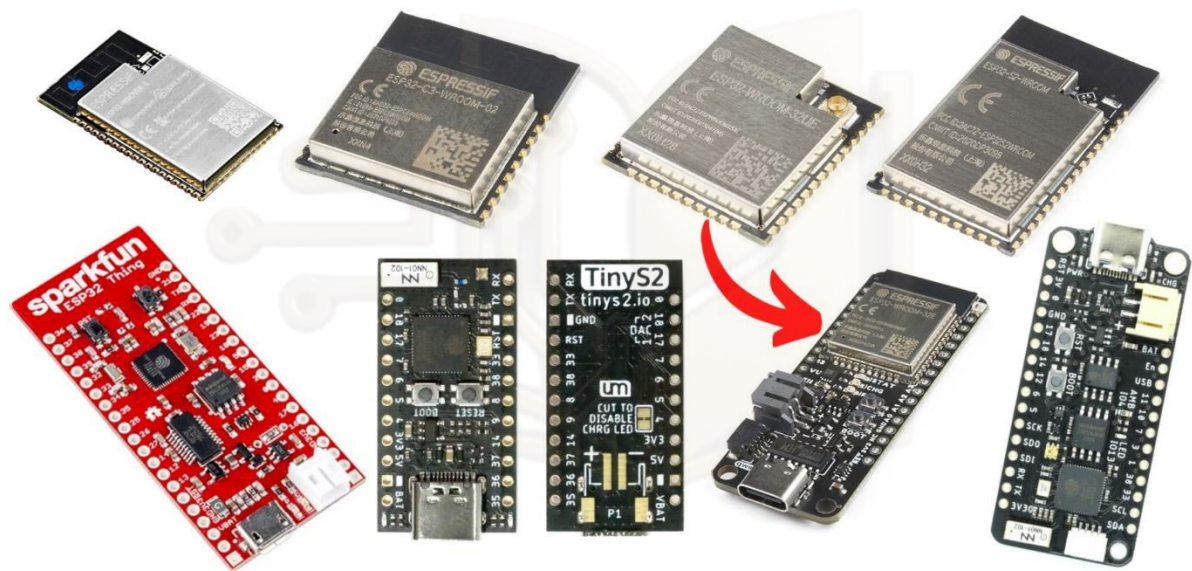


Figure 3-3: Different Version of ESP32

- **ESP32-WROOM-32:** The most popular module, offering general-purpose use with integrated WiFi and Bluetooth.
- **ESP32-S2:** A low-power version optimized for IoT applications that do not require Bluetooth.
- **ESP32-C3:** A cost-effective option with RISC-V architecture, supporting WiFi and BLE.
- **ESP32-S3:** Designed for AI and IoT, featuring enhanced computing power and additional GPIO pins.

Internet of Things (IoT) and Embedded

Unit 7

- **ESP32-PICO-D4:** A compact, all-in-one chip for space-constrained designs.
- **ESP32-WROVER:** Includes additional PSRAM, making it suitable for memory-intensive applications like image processing.

3.2.2 Detailed Features of ESP32-WROOM-32

The ESP32-WROOM-32 module is one of the most widely used boards in the ESP32 series. Its powerful dual-core processor and integrated wireless features make it ideal for IoT and smart devices.

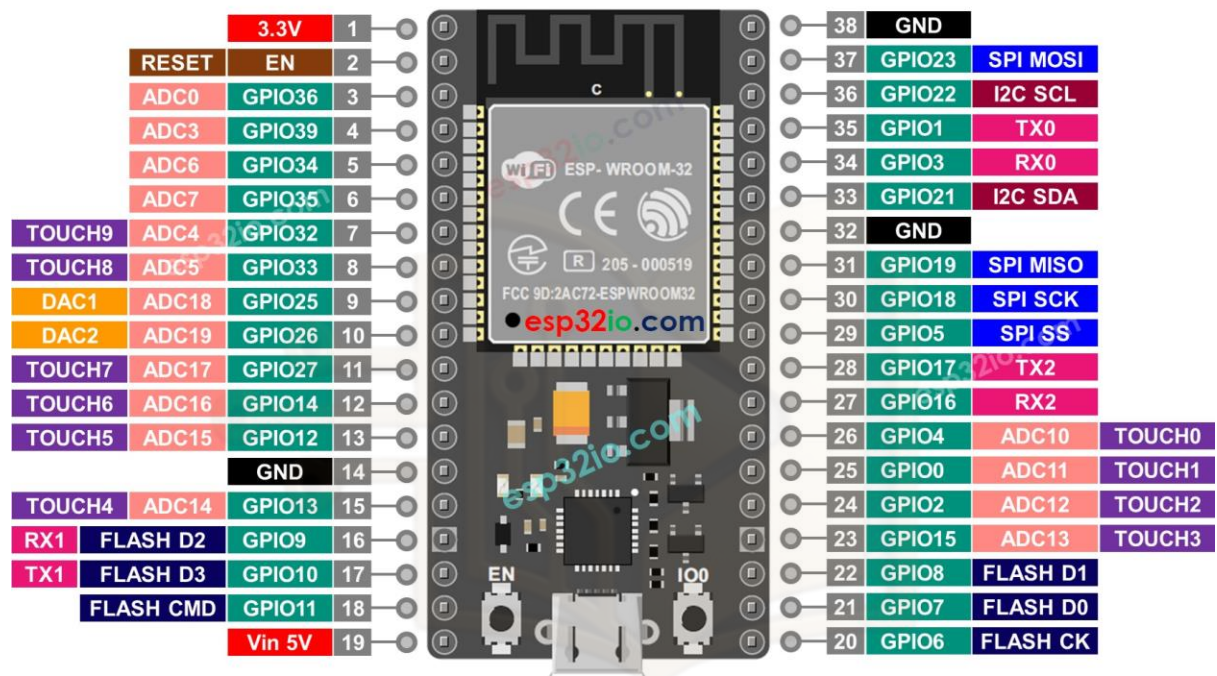


Figure 3.4: ESP32-WROOM-32 Pin Diagram

Key Features:

- **Microcontroller:** Tensilica Xtensa LX6 (dual-core).
- **Operating Voltage:** 3.3V.
- **Connectivity:**
 - WiFi: 802.11 b/g/n.
 - Bluetooth: Classic and BLE.
- **Clock Speed:** Up to 240 MHz.
- **Flash Memory:** 4 MB (or more, depending on the variant).
- **RAM:** 520 KB SRAM.

- **GPIO Pins:** Up to 39, configurable for digital, analog, PWM, I2C, SPI, and UART.
- **Power Consumption:** Ultra-low power, supports deep sleep modes.
- **Built-in Peripherals:** ADC, DAC, touch sensors, and PWM modules.

Capabilities:

- **Wireless Communication:** Seamless WiFi and Bluetooth integration for IoT devices.
- **Edge Processing:** Suitable for real-time data processing, including AI/ML tasks.
- **Wide Range of Applications:** Smart homes, wearable devices, and industrial automation.
- **Expandable Ecosystem:** Compatible with libraries and shields for additional functionality.

3.3 Raspberry Pi

The Raspberry Pi is a compact, affordable, and versatile single-board computer developed by the Raspberry Pi Foundation in the United Kingdom. Initially launched in 2012, it was designed to promote computer science education, particularly in schools and developing countries. Over time, it has gained immense popularity among hobbyists, developers, and researchers for its ability to handle diverse applications, including IoT, robotics, AI, and multimedia projects.

The Raspberry Pi stands out for its capability to function as a full-fledged computer while maintaining a small form factor and low cost. It supports various operating systems, with Raspberry Pi OS (formerly Raspbian) being the most commonly used. The platform's extensive community and open-source nature provide abundant resources for users of all skill levels.

3.3.1 Types of Raspberry Pi Models

Raspberry Pi boards come in several models, each catering to specific needs, ranging from basic learning and prototyping to advanced multimedia and industrial applications. Below are some popular Raspberry Pi models:

Internet of Things (IoT) and Embedded

Unit 7

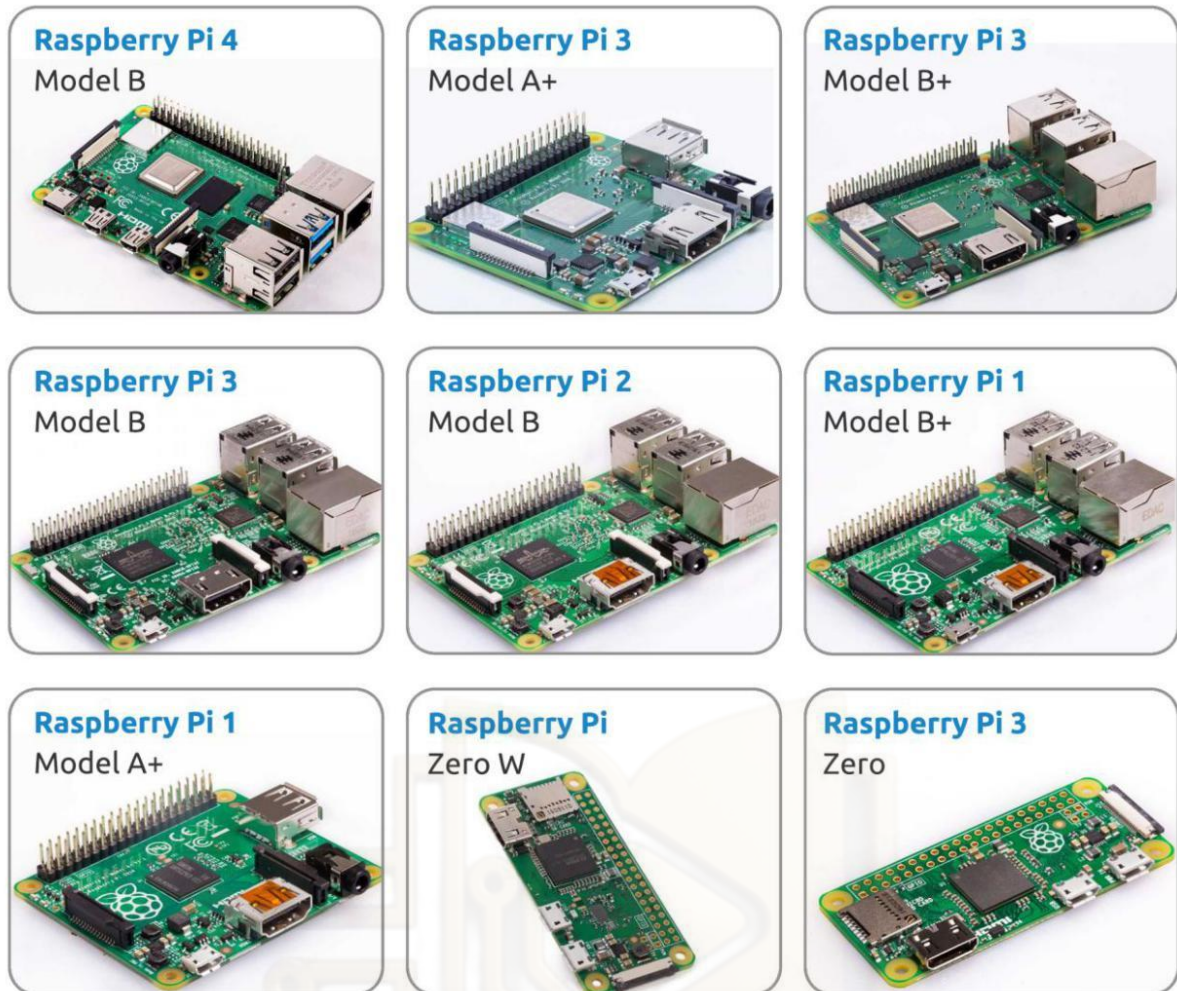


Figure 3-4: Some Popular Raspberry Pi Boards

- **Raspberry Pi 4 Model B:** High-performance model featuring up to 8 GB RAM, dual 4K display support, and USB 3.0 ports.
- **Raspberry Pi 3 Model B+:** Known for its integrated Wi-Fi and Bluetooth, suitable for moderate IoT and computing tasks.
- **Raspberry Pi Zero W:** A compact version with wireless capabilities, ideal for portable and low-power projects.
- **Raspberry Pi Pico:** A microcontroller board based on the RP2040 chip, tailored for embedded systems and IoT applications.
- **Raspberry Pi 400:** A keyboard-integrated computer, offering plug-and-play functionality for desktop use.

3.3.2 Detailed Features of Raspberry Pi 4 Model B

The **Raspberry Pi 4 Model B**, one of the most powerful and widely used models, is well-suited for advanced IoT projects, multimedia applications, and even light server tasks.

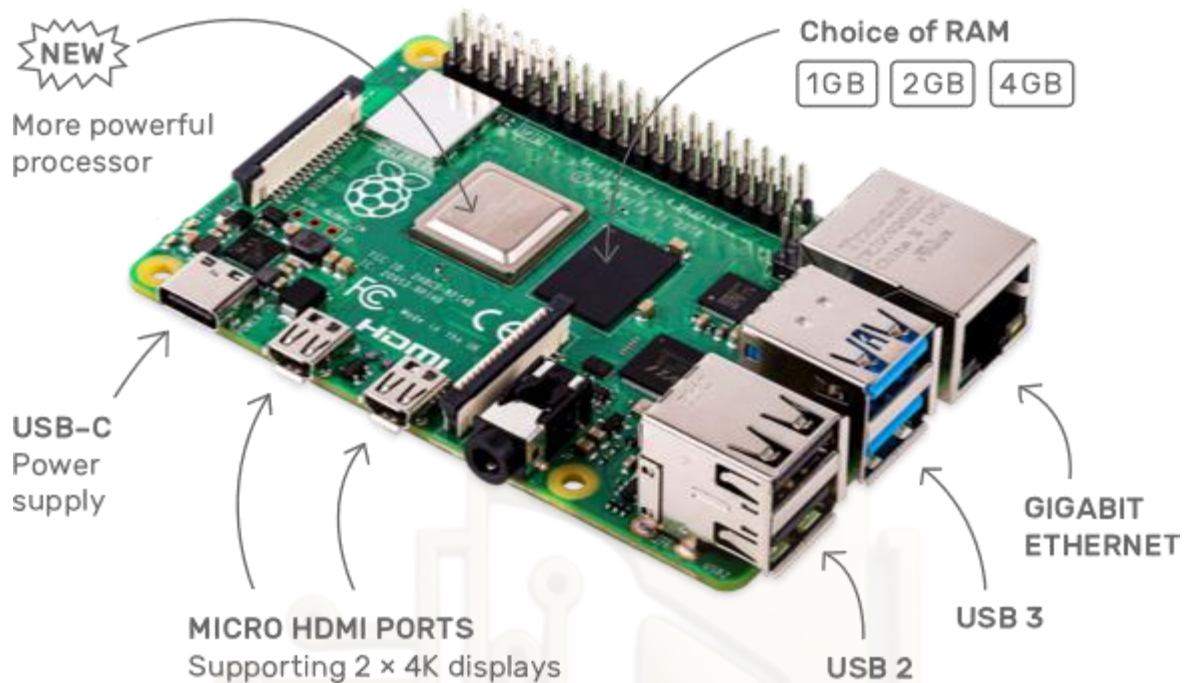


Figure 3-5: Raspberry Pi Pin-diagram

Key Features:

- **Processor:** Quad-core Cortex-A72 (ARM v8) 64-bit at 1.5 GHz.
- **Memory Options:** Available in 2 GB, 4 GB, and 8 GB LPDDR4 RAM variants.
- **Storage:** Boot from microSD card or USB storage.
- **Connectivity:**
 - 2 × USB 3.0 and 2 × USB 2.0 ports.
 - Gigabit Ethernet.
 - Dual-band Wi-Fi (2.4 GHz and 5 GHz).
 - Bluetooth 5.0.
 - Dual micro-HDMI ports with 4K display support.
- **GPIO:** 40-pin header for interfacing with sensors, actuators, and modules.
- **Power Supply:** USB-C input, requiring 5V/3A.
- **Operating Systems:** Compatible with Raspberry Pi OS, Ubuntu, and other Linux-based OSes.

Internet of Things (IoT) and Embedded

Unit 7

Capabilities:

- **IoT and Automation:** Ideal for smart home hubs, industrial automation, and edge computing.
- **Multimedia:** Supports dual 4K displays and hardware decoding for HEVC, making it perfect for media centers.
- **Prototyping and Development:** Wide GPIO support and compatibility with various software tools enable quick prototyping.
- **Educational Use:** Simplifies learning programming and computer science concepts.
- **Advanced Networking:** Gigabit Ethernet and dual-band Wi-Fi facilitate seamless connectivity for IoT systems.

4 Comparison of Arduino, ESP32, and Raspberry Pi

Arduino, ESP32, and Raspberry Pi are three of the most popular platforms for IoT and embedded systems projects, each catering to different requirements. While **Arduino** is known for its simplicity and ease of use in basic sensor-actuator projects, **ESP32** provides wireless connectivity for IoT applications. On the other hand, **Raspberry Pi** functions as a compact computer, suitable for complex and resource-intensive tasks. Below is a comparison of these platforms based on various criteria:

Feature	Arduino Uno	ESP32	Raspberry Pi
Type	Microcontroller	Microcontroller with integrated Wi-Fi/Bluetooth	Single-board computer
Operating System	None (runs directly on C/C++ programming)	Runs MicroPython, C/C++, JavaScript	Linux-based OS (e.g., Raspbian, Ubuntu)
Processor	ATmega328P, 8-bit, 16 MHz	Dual-core 240 MHz	Quad-core Cortex-A72, up to 1.5 GHz
Memory	2KB SRAM, 32KB FLASH, 1KB EEPROM	520KB SRAM, 4MB FLASH	2GB, 4GB, or 8GB RAM, microSD up to 32GB
Clock Frequency	16 MHz	Up to 240 MHz	1.5 GHz
Camera Port	No	No	CSI camera port
Input/Output Pins	14 digital I/O, 6 analog I/O	34 digital I/O, 18 analog I/O	40 GPIO pins
Power Consumption	200 mW	50 mW average	700 mW
Background Storage	None	4MB FLASH	microSD card slot
Price	\$10 - \$50	\$5 - \$15	\$35 - \$75
Other Features	Simple, low-power, fast response time	In-built Wi-Fi, Bluetooth, good for IoT	Multimedia capabilities, desktop-level tasks

Review Questions

- Q.1** What is the Internet of Things (IoT), and Explain IoT architecture with its main components?
- Q.2** Explain the role of embedded systems in the IoT ecosystem.
- Q.3** What is MQTT, and how does it work as an IoT communication protocol?
- Q.4** Explain the MQTT publish-subscribe model. How does it facilitate scalable communication between IoT devices?
- Q.5** What are the key features of MQTT that make it suitable for IoT applications?
- Q.6** What are the primary differences between Arduino, ESP32, and Raspberry Pi?
- Q.7** What unique features does Raspberry Pi offer that make it suitable for IoT projects that require complex data processing or a graphical user interface?
- Q.8** Write short note on:
- i Arduino Uno
 - ii Esp32
 - iii Raspberry pi
 - iv Short history on IoT